算法组招新任务实验报告

OpenCV 图像处理 · MLP 手写数字识别 · YOLO 无人机目标检测与量化

姓　名　_____

学　号　_____

班　级　_____

完成内容　task-2 / task-3 / task-4（题目一）

日　期　2026 年 10 月

摘 要

本报告完成了 2026 年算法组招新题目中的 task-2、task-3 与 task-4（题目一）。task-2 使用 OpenCV 与 Pillow 完成图像读写、通道分离、缩放/旋转/平移/翻转、目标轮廓检测、MOG2 与帧差法运动目标检测，以及 CSRT/KCF/MOSSE 三种相关滤波跟踪器的小球追踪实验；task-3 在理解深度学习基本概念的基础上，用 PyTorch 自动求导与梯度下降完成二元函数最小二乘拟合，并搭建 MLP 在 MNIST 上取得 98.62% 的测试准确率；task-4 题目一使用 YOLO11n 官方预训练权重完成 COCO 图像推理，基于无人机数据集与给定 drone.pt 微调出无人机检测模型，完成视频推理、mAP/混淆矩阵/速度指标分析，并进一步导出 ONNX、进行 FP16 与 INT8 量化，比较模型体积、精度与推理速度。

关键词：OpenCV；背景减除；目标跟踪；MLP；MNIST；YOLO11；无人机检测；模型量化；ONNX

1 环境配置（task-1）

task-1 要求完成 Python / CUDA / PyTorch 环境配置。本项目在 Windows 11、Python 3.13 下创建独立虚拟环境，安装 CUDA 12.8 版 PyTorch 与 ultralytics，并使用 NVIDIA GeForce RTX 4060 Laptop GPU（8 GB 显存）完成训练与推理。环境验证命令与结果如下：

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
>>> 2.11.0+cu128 True

python -c "import cv2; print(cv2.__version__)"
>>> 4.12.0 (opencv-contrib-python，含 CSRT/KCF/MOSSE 跟踪器)
```

表 1-1 实验环境与版本

组件	版本	说明
操作系统	Windows 11	训练/推理平台
Python	3.13.15	虚拟环境 C:\Users\outof\venvs\cvhw
PyTorch	2.11.0+cu128	CUDA 12.8，支持 AMP
torchvision	0.26.0+cu128	与 PyTorch 匹配
ultralytics	8.4.179	YOLO11 训练 / 导出
OpenCV	4.12.0（contrib）	图像处理与跟踪器
GPU	RTX 4060 Laptop 8 GB	CUDA 可用，torch.cuda.is_available()=True

此外，任务中所有脚本都使用 `pathlib` 基于文件位置定位项目根目录，因此可在任意工作目录下运行；涉及中文路径的图片使用 `cv2.imdecode` 读取，避免 Windows 下 `cv2.imread` 对非 ASCII 路径的兼容问题。

2 OpenCV 与 Pillow 图像处理（task-2）

2.1 图像读取、显示与三通道分离

OpenCV 的 `cv2.imread` 默认按 **BGR** 顺序读取彩色图，而 Pillow 按 **RGB** 顺序存储；两者读入的像素在颜色空间转换后应完全一致。实验先分别用两个库读取 example1.jpg，将 OpenCV 结果由 BGR 转为 RGB 后与 Pillow 结果逐像素比较，`np.array_equal` 返回 True，说明两库解码结果一致。随后用 `cv2.split` 分离 B/G/R 三个通道，并分别以灰度图和“单通道彩色化”两种方式可视化。

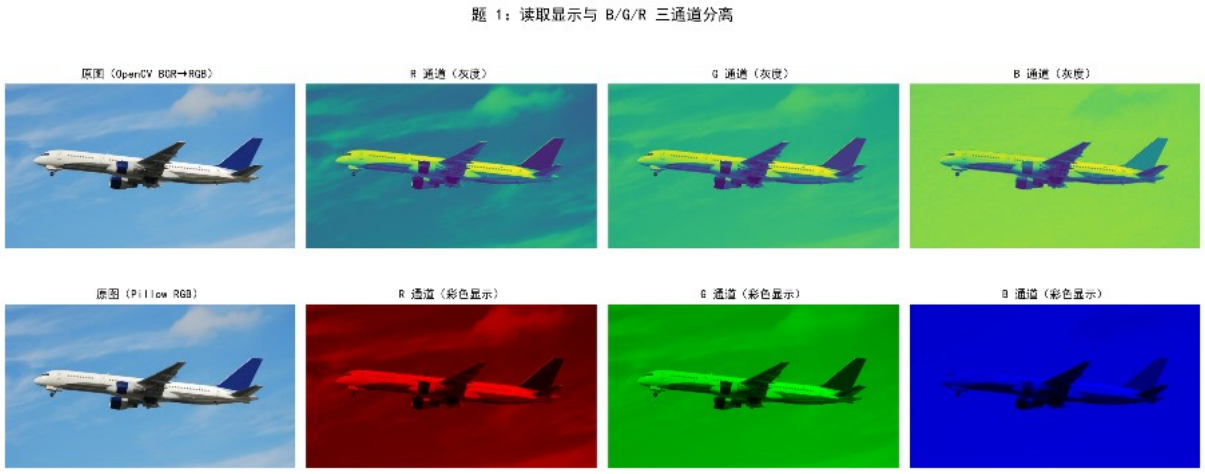


图 2-1 OpenCV/Pillow 读图与 B/G/R 通道分离结果

从图 2-1 可以看到：R 通道中机身白色区域最亮、天空较暗，B 通道中天空最亮、机身偏暗，G 通道介于两者之间。这正是“白色机身 + 蓝色天空”的颜色分布，也为 2.3 节利用颜色先验分割飞机轮廓提供了依据。

2.2 缩放、旋转、平移与翻转

几何变换的本质是像素坐标的映射与重采样。设原图像坐标为 (x, y)，变换后坐标为 (x’, y’)，四种操作可统一用仿射变换描述：

$$[[x'], [y']] = M \cdot [[x], [y], [1]], \quad M = [[a11, a12, tx], [a21, a22, ty]]$$

缩放通过 `cv2.resize` 实现，缩小使用 `INTER_AREA`（对区域像素求平均，可减轻摩尔纹），放大使用 `INTER_CUBIC`（三次插值，边缘更平滑）。**旋转**使用 `cv2.getRotationMatrix2D(center, angle, scale)` 生成绕中心旋转的仿射矩阵；为避免旋转后四角被裁掉，实验根据旋转角重新计算外接画布尺寸，并修正平移量，使旋转后的图像完整落在新画布内。**平移**直接在仿射矩阵的平移分量上加上 (tx, ty)。**翻转**使用 `cv2.flip`，参数 1 表示水平翻转、0 表示垂直翻转、-1 表示同时翻转。

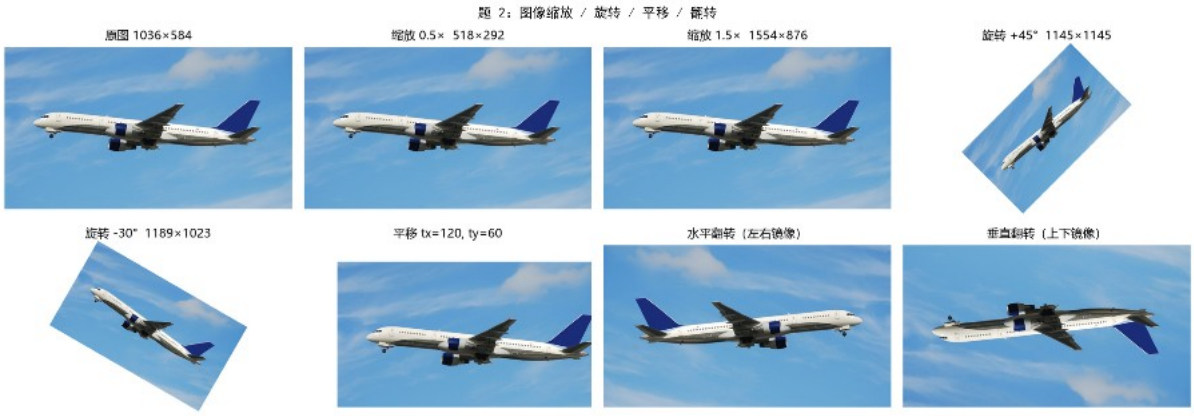


图 2-2 缩放、旋转（扩展画布）、平移与翻转结果

表 2-1 几何变换参数与输出尺寸

操作	输出尺寸	插值/边界	说明
原图	1036×584	—	—
缩放 0.5×	518×292	INTER_AREA	面积变为 1/4，用于加速预览
缩放 1.5×	1554×876	INTER_CUBIC	放大后细节更平滑
旋转 +45°	1145×1145	双线性 + 白边	扩展画布，四角不丢失
旋转 -30°	1189×1023	双线性 + 白边	逆时针旋转
平移 (120, 60)	1036×584	仿射平移	空白区域填白
水平/垂直翻转	1036×584	cv2.flip	镜像变换

需要注意，旋转 +45° 时 OpenCV 输出 1145×1145，而 Pillow `rotate(45, expand=True)` 输出 1146×1146，二者相差 1 像素，来源于画布尺寸取整方式不同。这类差异在图像拼接等任务中需要统一坐标系与取整规则，否则会出现 1 像素错位。

2.3 轮廓检测与面积、周长统计

example1.jpg 中飞机为白色/深蓝色机身，背景为高饱和蓝色天空。若直接对灰度图做阈值分割，天空与白色机身亮度接近，难以分开；因此实验采用**颜色先验**：在 BGR 空间计算 B−R 色差，天空区域 B 远大于 R，而白色机身 B≈R，从而得到天空掩膜，取反即前景。随后限定目标所在的水平带（去除上下边缘云层），用闭运算连接机身、机翼、尾翼与发动机，用开运算去除噪点，取最大连通域并填充内部孔洞，最后用 `cv2.findContours` 提取外轮廓。

轮廓面积由格林公式计算（OpenCV `cv2.contourArea`），周长由相邻轮廓点欧氏距离累加（`cv2.arcLength(contour, closed=True)`）。为便于比较，实验同时计算了凸包与多边形近似（Douglas－Peucker， ε =1% 周长）的面积/周长。

表 2-2　飞机轮廓面积与周长统计

轮廓	面积 / px²	周长 / px	点数	占图像面积
外轮廓（整机剪影）	71326.5	2197.7	1967	11.79%
凸包	114174.5	1800.4	29	—
多边形近似（ε =1%P）	71827.0	1976.6	14	—

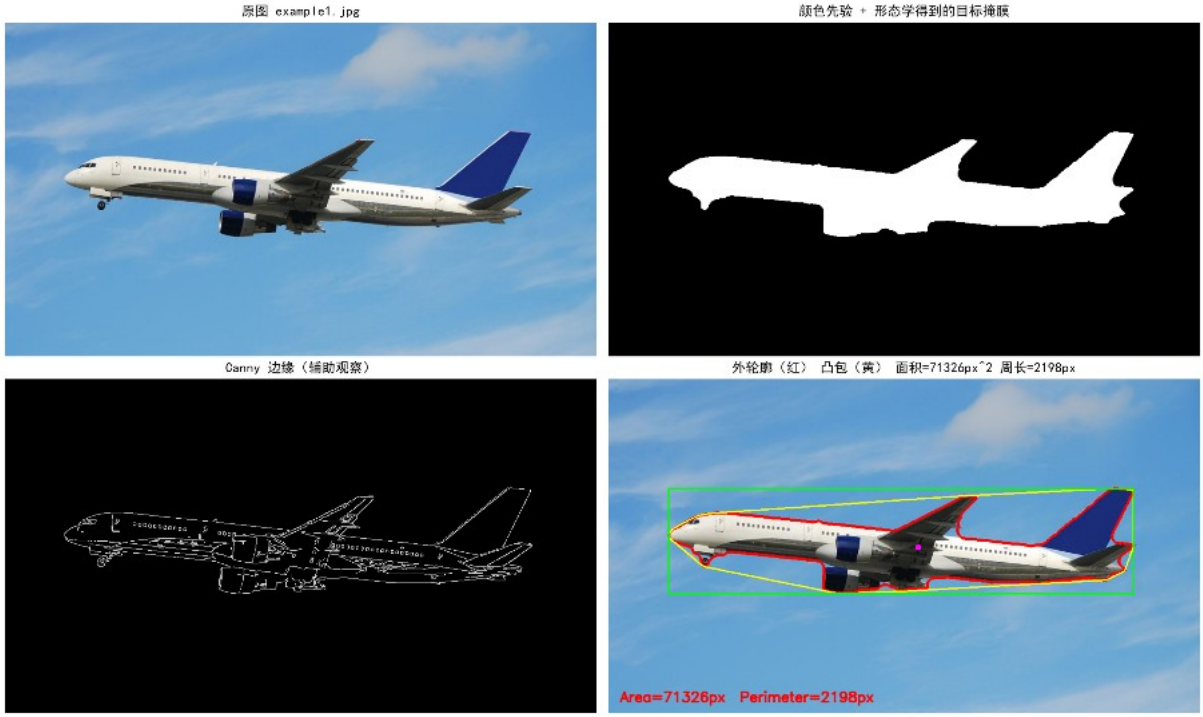


图 2-3　颜色先验分割、Canny 边缘与轮廓标注结果

实验得到整机外轮廓面积 **71,326 像素²**、周长 **2,198 像素**，占图像面积的 11.79%，外接矩形为 105×191 起、宽 815、高 184。凸包面积为 114174.5 像素²，明显大于真实轮廓面积，这是因为机身与机翼、尾翼之间存在较大的凹形空隙，凸包会把这些空隙一并包住；而 14 个顶点的多边形近似面积仅比原轮廓多约 0.7%，说明 1% 周长的 ε 已能较好地保留飞机外形。

轮廓检测的关键在于**分割质量**而非轮廓函数本身：若天空阈值过松，会把白色云层并入前景；若过紧，则机身暗部会断裂。本实验通过“B-R 色差 + 水平带约束 + 最大连通域 + 填孔”组合，使轮廓在不需要人工框选的情况下保持完整。

2.4 运动目标检测：MOG2 与帧差法对比

example2.mp4 是一段固定机位拍摄的高速公路视频（1280×720，29.97 fps，984 帧）。实验比较两类方法：

- **MOG2 背景减除**：为每个像素建立 K 个高斯分布的混合模型，按像素值出现的频率自适应更新；当前帧像素与背景模型差异较大时判为前景。MOG2 还能标记阴影（灰度 127），实验中将其剔除。
- **帧差法（无背景模型）**：用当前帧与前一帧的灰度差 `|I_t - I_{t-1}|` 做阈值分割。它不维护背景，只对“发生变化”的像素敏感，是典型的不使用背景减除的基线方法。

$$\text{MOG2: } p(x_t) = \Sigma_\{k=1..K\}\ \omega_\{k,t\} \cdot N(x_t \mid \mu_\{k,t\}, \Sigma_\{k,t\})$$

$$\text{帧差法: } D_t(x,y) = 1,\ \text{if } |I_t(x,y) - I_\{t-1\}(x,y)| > T;\ \text{else } 0$$

两种方法均使用 3×3 开运算去除噪点、7×7 闭运算连接断裂区域，并过滤面积小于 300 像素的轮廓；检测框画在原视频上，同时保存各自的掩膜对比视频。

表 2-3 MOG2 与帧差法的运动检测指标对比

指标	MOG2 背景减除	帧差法（无背景减除）
平均检测目标数 / 帧	6.03	6.21
最多检测目标数	14	16
平均前景像素占比	2.31%	2.41%
平均检测框填充率（轮廓面积/框面积）	0.605	0.484
平均目标框面积 / px²	3368	3556
相邻帧掩膜 IoU（时间稳定性）	0.564	0.335



图 2-4　MOG2 与帧差法在四个关键帧上的前景掩膜对比

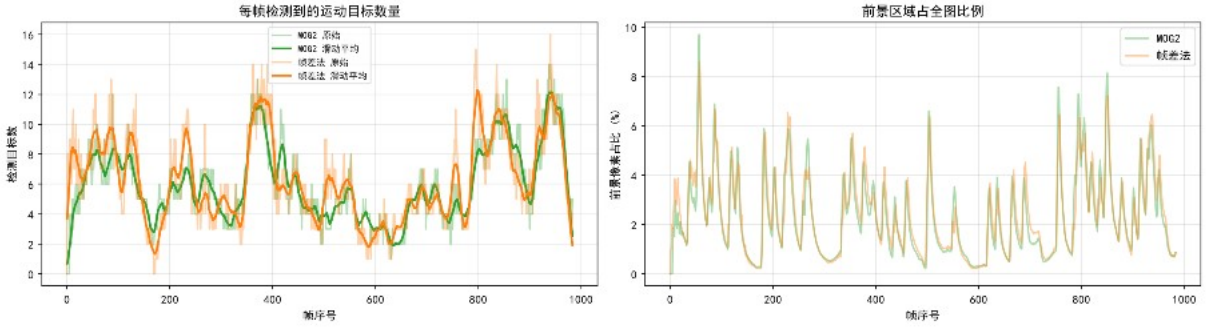


图 2-5　两种方法的逐帧检测目标数与前景像素占比

从掩膜和指标可以看出：帧差法得到的车辆掩膜呈“空心/双边”形状，因为匀速运动的车身内部纹理在相邻帧间变化很小，只有车身边缘产生较大差分；其检测框填充率仅 0.48，相邻帧 IoU 只有 0.33，说明目标轮廓不完整且随时间抖动。MOG2 维护了背景模型，能给出较完整的车身区域，填充率达到 0.60，时间稳定性也更高（IoU 0.56）。

但 MOG2 也有明显不足：第一，车辆长时间停留或减速时会被逐渐吸收进背景，导致漏检；第二，背景模型更新速度与车速、光照变化需要权衡；第三，遇到摄像机抖动或大面积光照突变时会出现整片“鬼影”。帧差法则完全无法检测静止目标，且目标运动越慢，差分越弱。实际工程中常将两者结合：用 MOG2 获取完整前景，用帧差法提供瞬时运动证据，再通过形态学与跟踪器做后处理。

2.5 小球跟踪：CSRT / KCF / MOSSE 与检测辅助跟踪

task-2 第 5 题要求用 OpenCV 跟踪器追踪 example3.mp4 与 example4.mp4 中的小球。实验选择三种经典相关滤波跟踪器：

- **CSRT**（Discriminative Correlation Filter with Channel and Spatial Reliability）：在 HOG + 颜色特征上学习判别相关滤波器，并用空间可靠性掩膜抑制背景干扰；精度高，但速度较慢。
- **KCF**（Kernelized Correlation Filter）：利用循环矩阵与高斯核把相关运算转到频域，速度快，但对尺度变化和遮挡较敏感。
- **MOSSE**（Minimum Output Sum of Squared Error）：只用灰度特征，以最小输出平方误差求解滤波器，速度极快（可达数百至上千 FPS），精度最低。

对 example3（纯色背景上的红色小球），用 HSV 红色阈值检测框作为参考真值，统计跟踪框与参考框的 IoU（≥0.5 视为成功）；对 example4（白色乒乓球、手持移动、镜头运动、手部遮挡），由于没有逐帧真值，采用“模板外观相关分数”作为跟踪有效性的替代指标，并记录跟踪器丢失帧数与重初始化次数。

表 2-4 example3 红色小球跟踪结果

方法	成功率 (IoU≥0.5)	平均 IoU	跟踪器丢失帧	处理 FPS
CSRT	100.00%	0.9099	0	27.7
KCF	100.00%	0.9023	0	204.4
MOSSE	100.00%	0.9059	0	903.9
CSRT+颜色重检测 +Kalman	100.00%	0.9297	0	26.6

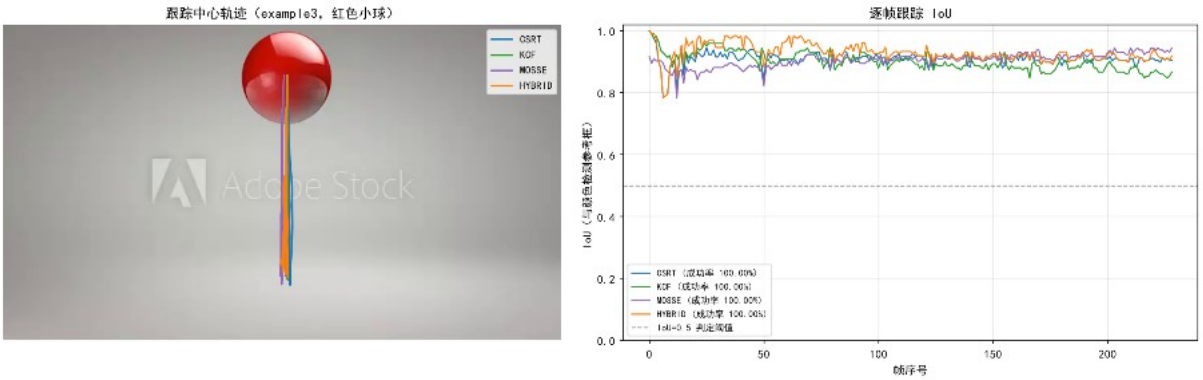


图 2-6 example3 跟踪中心轨迹与逐帧 IoU

example3 背景干净、目标颜色鲜明，三种跟踪器均保持 100% 成功率。MOSSE 速度最快（约 904 FPS），但平均 IoU 略低；CSRT 精度最高但只有约 28 FPS；KCF 在精度与速度之间较均衡（约 204 FPS）。混合方法在 CSRT 基础上加入颜色检测校正与 Kalman 平滑，平均 IoU 提升到 0.930，但额外检测使速度降到约 27 FPS。这一组结果说明：**在目标外观稳定、背景简单时，传统相关滤波跟踪器已经足够，选择的主要依据是速度/精度权衡**。

表 2-5 example4 白色乒乓球跟踪结果

方法	有效率（外观分≥0.45）	平均外观分	丢失/预测帧	重初始化次数	处理 FPS
纯 CSRT	45.96%	0.3571	498	0	38.5
模板匹配+颜色约束+CSRT+Kalman	59.55%	0.4118	211	50	5.7

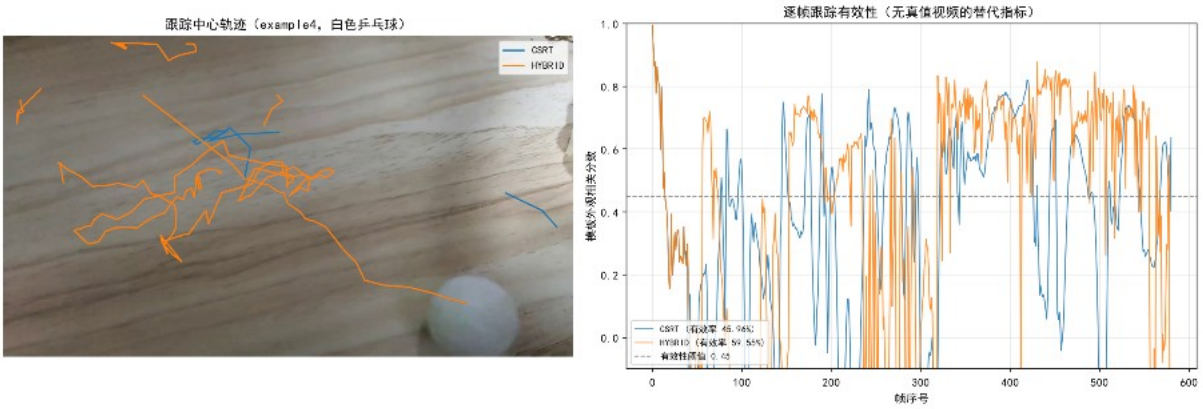


图 2-7 example4 跟踪中心轨迹与逐帧外观相关分数

example4 的难度远高于 example3：目标是无纹理的白色乒乓球，画面中存在白色杯盖、瓷盘等相似外观物体；手持移动导致目标快速平移和尺度变化；手部会短时遮挡目标；镜头本身也在运动。纯 CSRT 在初始阶段尚能跟上，但一旦被手遮挡或目标快速移动，就会丢失并停在错误位置，有效率仅 46.0%。实验进一步实现了一个“模板匹配 + 蓝白色先验 + 尺度连续性 + CSRT 重初始化 + Kalman 预测”的混合跟踪器：在预测位置附近做多尺度模板匹配，用蓝白色像素比例抑制木桌和暖色背景，用尺度突变惩罚抑制跳到杯盖，跟踪器失败时用检测框重置，连续丢失时用 Kalman 外推。该方法把有效率提升到 59.6%，但仍有约 211 帧处于预测/丢失状态，且模板匹配每帧需遍历多个尺度，速度只有约 5.7 FPS。

如何提高跟踪准确度？ 结合本实验的失败案例，可以从五个方面改进：（1）**检测辅助跟踪**：用颜色/形状/Hough 圆检测或轻量目标检测器周期性重检测，替代纯模板匹配，避免白色杯盖等相似物体误匹配；（2）**运动模型**：用 Kalman/粒子滤波预测目标位置，缩小搜索范围，并在遮挡时保持轨迹连续；（3）**多特征融合**：把颜色直方图、圆形度、尺度先验与相关滤波响应融合，降低无纹理目标的漂移；（4）**深度跟踪器**：如 DaSiamRPN、TrackerViT 等孪生网络跟踪器对无纹理目标更鲁棒，但需要 GPU；（5）**评估与数据**：对 example4 手动标注少量关键帧，计算成功率/精度（Precision）与 EAO，才能客观比较不同方法，而不是仅凭肉眼观察。

3 MLP 识别 MNIST（task-3）

3.1 深度学习基本概念理解

神经元与感知机。 神经元是神经网络的基本计算单元：它对输入向量 x 做加权求和并加上偏置 b，再经过非线性激活函数 f 得到输出 a = f(w • x + b)。单个感知机只能表示线性可分函数，多层堆叠后才能逼近复杂非线性映射。

权重与偏置。 权重 w 决定各输入维度的重要性与方向，偏置 b 控制激活阈值，使决策边界不必经过原点。训练的过程就是不断调整 w、b 使损失下降。

激活函数。 没有激活函数时，多层线性变换仍等价于一个线性变换。Sigmoid 把输出压到(0,1)，但两端梯度饱和、易梯度消失；Tanh 以 0 为中心，仍有饱和问题；ReLU（max(0,x)）计算简单、正区间梯度恒为 1，是深层网络的默认选择，但可能出现“神经元死亡”；LeakyReLU、GELU、SiLU 等是其改进版本。

前向传播。 输入依次经过各层线性变换与激活，得到预测值。以两层网络为例：h = f(W1 • x + b1)，$\hat{y}$ = g(W2 • h + b2)。前向传播只做计算、不更新参数。

隐藏层、输出层与全连接层。 隐藏层负责逐级抽取特征，输出层把特征映射到任务空间（分类任务常用 softmax 得到类别概率）。全连接层中每个神经元与上一层所有神经元相连，参数量大但表达能力强；卷积层则通过局部连接与权值共享减少参数，更适合图像。

损失函数。 回归常用均方误差 MSE = $(1/n)\Sigma(\hat{y}-y)^2$；分类常用交叉熵 $L = -\Sigma\ y_c \log p_c$，它衡量预测分布与真实分布的差异，配合 softmax 时梯度形式简洁、收敛稳定。

梯度下降。 沿损失对参数的负梯度方向更新：$w \leftarrow w - \eta \cdot \partial L/\partial w$。按每次使用的样本量分为批量梯度下降（全量，稳定但慢）、随机梯度下降（单样本，快但抖）与小批量梯度下降（折中，最常用）。学习率 η 过大会震荡甚至发散，过小则收敛缓慢。

反向传播与链式法则。 反向传播利用链式法则从输出层向输入层逐层计算损失对每个参数的偏导：$\partial L/\partial W1 = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial h \cdot \partial h/\partial W1$。PyTorch 的 autograd 在前向时记录计算图，反向时自动完成这一过程，无需手写导数。

优化器。 SGD 只沿当前梯度更新；Momentum 引入历史梯度动量，抑制震荡、加速收敛；RMSProp/Adam 为每个参数自适应调整学习率；AdamW 把权重衰减与梯度更新解耦，是 Transformer/CNN 常用的默认优化器。

Epoch、Batch、Iteration。 一个 epoch 表示完整遍历一次训练集；batch 是一次前向/反向使用的样本数；iteration 是完成一个 batch 的参数更新次数，因此 iteration = epoch × (训练样本数 / batch size)。

过拟合与欠拟合。 欠拟合指模型在训练集和验证集上都表现差，通常因为模型容量不足或训练不充分；过拟合指训练集表现很好、验证集明显变差，模型记住了训练样本的噪声。可通过观察 Loss/Accuracy 曲线的分叉判断。

正则化。 L1 正则倾向于产生稀疏权重，可用于特征选择；L2 正则抑制权重过大、使模型更平滑；Dropout 在训练时随机失活部分神经元，迫使网络学习冗余特征；早停（Early Stopping）在验证集指标不再提升时停止训练；数据增强（翻转、裁剪、颜色扰动）则从数据层面提升泛化能力。

批归一化（Batch Normalization）。 对一个 mini-batch 内每个通道做标准化：$\hat{x} = (x-\mu_B)/\sqrt{(\sigma_B^2+\varepsilon)}$，再学习缩放 γ 和平移 β。BN 使各层输入分布稳定，允许更大学习率、加速收敛，并有一定正则效果；推理时使用训练阶段累积的滑动均值/方差。

训练集/验证集/测试集。 训练集用于更新参数；验证集用于选择超参数和早停；测试集只在最后评估一次，用来估计泛化误差。若用测试集调参，会造成信息泄漏，测试指标会偏乐观。

Tensor 与自动求导。 Tensor 是 PyTorch 的多维数组，支持 GPU 加速；设置 `requires_grad=True` 后，PyTorch 会记录在其上执行的所有运算，调用 `.backward()` 即可自动计算梯度，梯度保存在 `.grad` 中。

GPU 与 CPU 的区别。 CPU 核心少但单核性能强、擅长串行逻辑；GPU 拥有数千个流处理器，擅长大规模矩阵/卷积并行计算。深度学习中 GPU 的吞吐量通常是 CPU 的数十倍，但显存有限，需要控制 batch size 与模型规模。

混合精度训练。 用 FP16/BF16 存储和计算大部分前向/反向运算，同时保留 FP32 主权重与损失缩放（GradScaler），可在几乎不损失精度的前提下减少显存占用、提升吞吐量。PyTorch 通过 `torch.amp.autocast` 与 `GradScaler` 实现。

3.2 梯度下降最小二乘拟合

题目给定二元函数：

$$y = 2x_1^2 + 1.5x_1x_2 + 3\sin(x_1) + 0.5\cos(2x_2) + e\text{^}(-0.5x_1) + 0.8x_2 + 2026 + \varepsilon,\ \varepsilon \sim U(-3,3)$$

将其写成带未知参数的拟合形式：

$$\hat{y} = a_1x_1^2 + a_2x_1x_2 + a_3\sin(x_1) + a_4\cos(2x_2) + e\text{^}(a_5x_1) + a_6x_2 + b$$

在 $x_1, x_2 \in [-3, 3]$ 内随机采样 4000 个点，加入 U(-3,3) 噪声；参数初始化为 $a_1=a_2=a_3=a_4=a_6=1$、$a_5=-0.2$、$b=0$。第一阶段用 Adam（学习率 0.05，余弦退火，4000 步）最小化 MSE；由于 $\exp(a_5x_1)$ 对 a_5 非线性，Adam 后期收敛变慢，第二阶段用 LBFGS（强 Wolfe 线搜索，200 次迭代）精调。所有梯度均由 PyTorch autograd 自动计算。

表 3-1 最小二乘拟合参数对比

参数	真值	拟合值	绝对误差	相对误差
a1	2	2.013078	0.013078	0.654%
a2	1.5	1.502036	0.002036	0.136%
a3	3	2.939890	0.060110	2.004%
a4	0.5	0.462230	0.037770	7.554%
a5	-0.5	-0.488969	0.011031	2.206%
a6	0.8	0.759977	0.040023	5.003%
b	2026	2026.019981	0.019981	0.001%

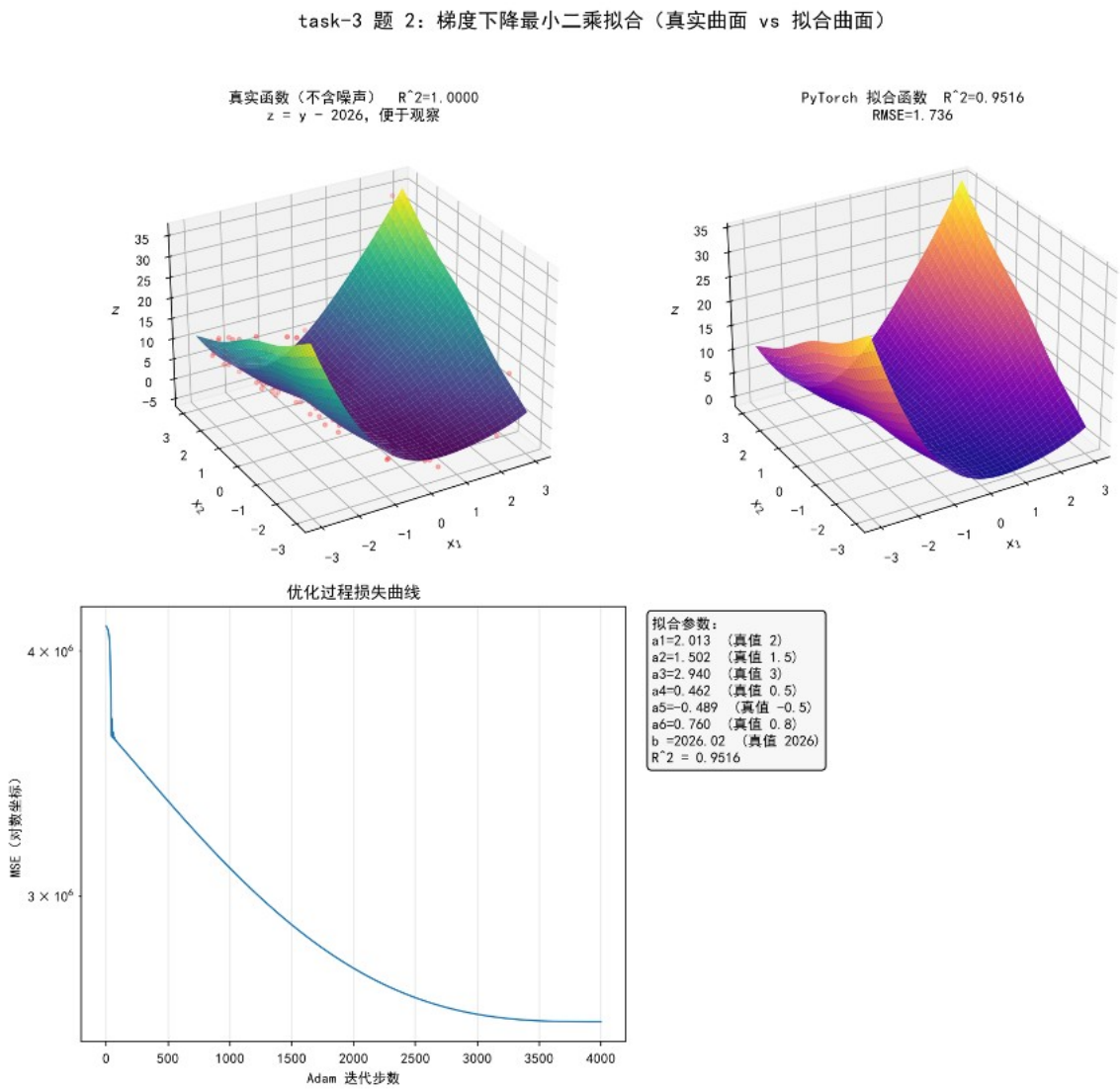


图 3-1 真实曲面、PyTorch 拟合曲面与优化损失曲线

拟合结果为 **R^2 = 0.9516**、RMSE = 1.7364、MAE = 1.5046。噪声均匀分布在 ±3 内，其理论标准差为 $3/\sqrt{3} \approx 1.732$，与 RMSE 几乎一致，说明模型已经拟合到噪声水平，剩余误差主要来自噪声而非模型偏差。各参数相对误差均小于 8%，其中 b 的误差仅 0.001%，a_1、a_2、a_5 的误差在 2% 以内；a_4 的相对误差最大（约 7.6%），因为 $\cos(2x_2)$ 项幅值只有 0.5，且与常数项、x_2 项存在一定共线性，噪声对其估计影响更明显。

为了便于观察，图中 z 轴绘制的是 y−2026 后的曲面；若直接把 2026 画进去，曲面会整体抬到 2000 以上，函数形状被压缩得难以辨认。R^2 对常数平移不变，因此这种可视化方式不影响指标含义。

3.3 MLP 识别 MNIST

MNIST 训练数据以 parquet 格式提供（60000 张 28×28 PNG，字段 image/label）。实验用 pandas 读取 parquet，用 OpenCV 解码 PNG 字节，按 8:1:1 分层划分为训练集 48000、验证集 6000、测试集 6000，保证各类别比例一致；输入按 MNIST 全局均值 0.1307、标准差 0.3081 标准化。

模型为 784→512→256→10 的全连接 MLP，每个隐藏层后接 BatchNorm、ReLU 与 Dropout(0.2)。总参数量 537,354。训练配置：AdamW（lr=1e-3，weight_decay=1e-4）、余弦退火、batch size 256、20 epochs、交叉熵损失、CUDA 混合精度（AMP）。在 RTX 4060 上训练 20 轮仅需约 33 秒。

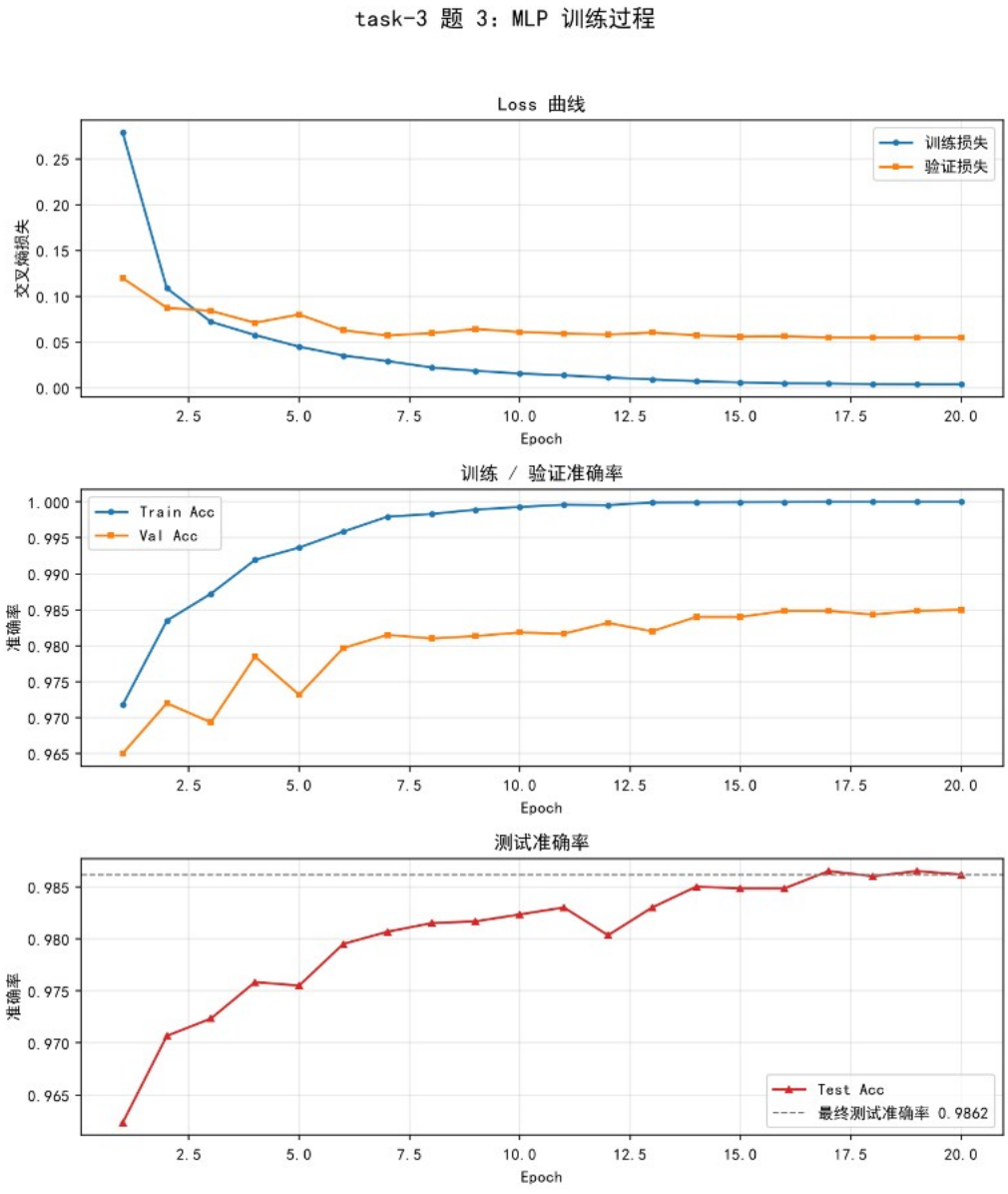


图 3-2 MLP 训练的 Loss、Train Acc 与 Test Acc 曲线

表 3-2 MLP 结构与训练配置

项目	配置/结果	参数量
输入	28×28 灰度图（Flatten 为 784）	—
隐藏层 1	Linear(784,512) + BN + ReLU + Dropout(0.2)	401,920
隐藏层 2	Linear(512,256) + BN + ReLU + Dropout(0.2)	131,328
输出层	Linear(256,10)	2,570
总参数量	—	537,354
优化器	AdamW，lr=1e-3，weight_decay=1e-4	—
训练轮次	20 epochs，batch=256，余弦退火	—
测试准确率	98.62%	—

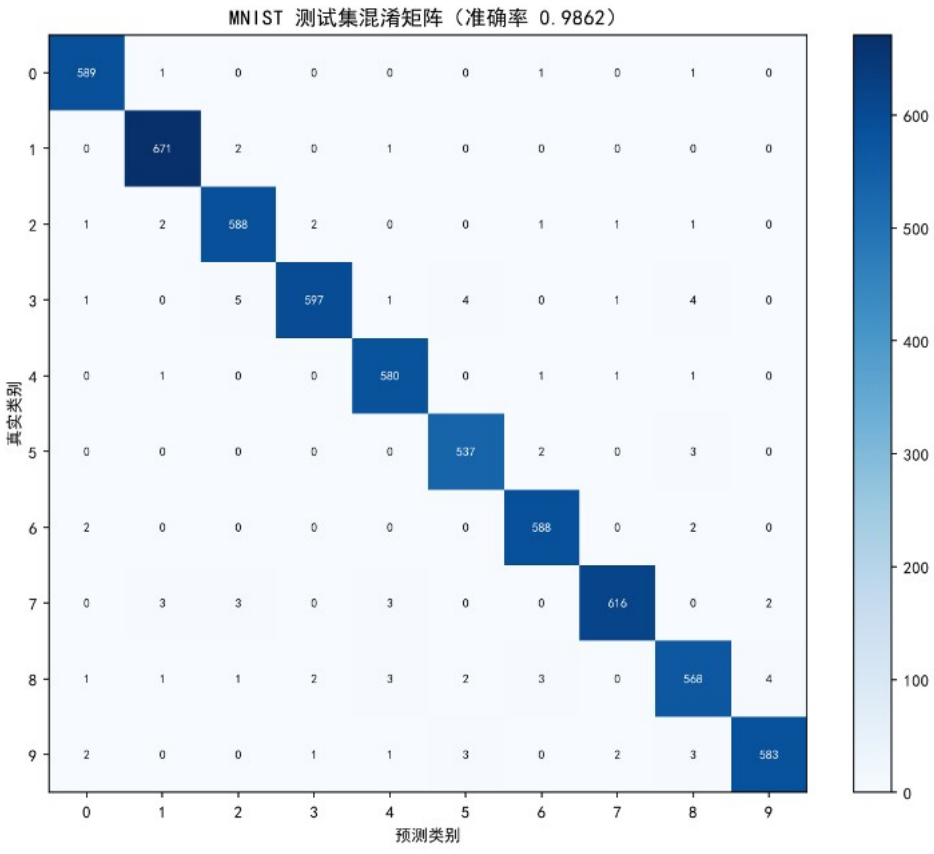


图 3-3 MNIST 测试集混淆矩阵

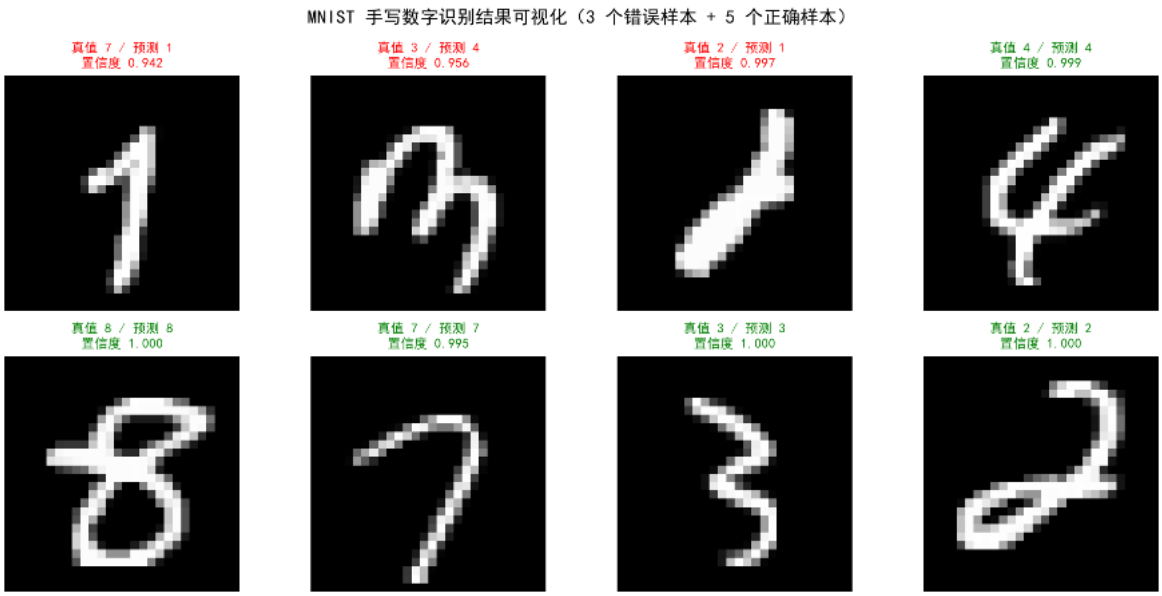


图 3-4 测试集预测可视化（含 3 个错误样本）

表 3-3 MNIST 各类别分类指标

数字	Precision	Recall	F1-score	测试样本数
0	0.9883	0.9949	0.9916	592
1	0.9882	0.9955	0.9919	674
2	0.9816	0.9866	0.9841	596
3	0.9917	0.9739	0.9827	613
4	0.9847	0.9932	0.9889	584
5	0.9835	0.9908	0.9871	542
6	0.9866	0.9932	0.9899	592
7	0.9919	0.9825	0.9872	627
8	0.9743	0.9709	0.9726	585
9	0.9898	0.9798	0.9848	595

最终测试准确率为 **98.62%**，宏平均 F1 为 0.9861。训练准确率在第 15 轮后达到 100%，而验证/测试准确率稳定在 98.5% 左右，存在约 1.5 个百分点的泛化间隙，说明模型出现了轻度过拟合；Dropout、BatchNorm 与 weight decay 已起到抑制作用。从混淆矩阵看，主要错误集中在 8↔3、7↔1、2↔1、9↔8 等形状相近的类别；其中数字 8 的 F1 最低（0.9726），因为 8 的上下两个闭合环在低分辨率下容易与 3、9 混淆。可视化中的三个错误样本也验证了这一点：书写倾斜的 7 被识别为 1，带连笔的 3 被识别为 4，顶部开口的 2 被识别为 1。

若继续提升准确率，可考虑：对训练集做随机平移/旋转/弹性形变增强；改用小型 CNN（如两层卷积 + 全连接）利用局部空间结构；增加标签平滑或 Cosine 学习率重启；用集成模型或测试时增强（TTA）。在 MNIST 上，MLP 的 98.6% 已接近该结构的性能上限，进一步提升空间主要来自卷积结构与数据增强。

4 YOLO 无人机目标检测与模型量化（task-4 题目一）

4.1 背景与 YOLO 原理

YOLO（You Only Look Once）把目标检测建模为单阶段回归：输入图像被划分为网格，每个网格直接预测若干目标的类别与边界框，一次前向即可得到检测结果。与 R-CNN 系列“先候选区域再分类”的两阶段方法相比，YOLO 速度更快，适合视频监控、无人机视觉等实时场景。

以本实验使用的 YOLO11n 为例，网络由三部分组成：**Backbone**（C3k2 模块 + SPPF）逐级下采样提取多尺度特征；**Neck**（PAN-FPN）把深层语义特征与浅层细节特征双向融合；**Head** 为解耦检测头，分别预测分类分数与边界框回归量。YOLO11 采用 anchor-free 设计，每个特征点直接回归到边界框四边距离，并用 DFL（Distribution Focal Loss）把距离建模为离散分布，提升边界回归精度。

训练时的正负样本分配使用 Task-Aligned Assigner（TAL），综合分类分数与预测框 IoU 选择正样本；损失由三部分组成：边界框回归损失（CIoU/DFL）、分类损失（BCE）与目标置信度损失。推理时经过置信度阈值筛选与 NMS（非极大值抑制）得到最终检测框。检测任务通常用 Precision、Recall、mAP@0.5 与 mAP@0.5:0.95 评价，而不是分类任务的 accuracy：mAP 同时考虑定位精度与类别正确性，对检测任务更合理。

4.2 官方预训练模型推理

首先使用 Ultralytics 官方 COCO 预训练权重 **yolo11n.pt**（80 类，约 5.4 MB，2.62 M 参数）对 example 目录下的三张图片推理，置信度阈值 0.25、输入 640×640。这一步用于熟悉 YOLO 的 API、输出格式与可视化方式，同时说明 COCO 预训练模型并不包含“无人机”类别，必须用无人机数据微调。

表 4-1 COCO 预训练模型在三张示例图上的检测结果

图片	尺寸	检测数	主要检测结果（类别/置信度）
coco-bear.jpg	586×640	1	bear(0.96)
coco-bus.jpg	640×425	8	bus(0.94)、person(0.52)、person(0.41)、traffic light(0.36)、car(0.35)、car(0.27)
coco-person.jpg	640×425	3	person(0.92)、skis(0.67)、skis(0.30)



图 4-1 YOLO11n 官方预训练权重推理结果

三张图分别检测到熊、公交车/行人/交通灯/汽车、以及滑雪者与滑雪板，置信度与定位均合理。公交车图中还出现了多个 person、traffic light、car 等 COCO 类别，说明预训练模型具备较强的通用特征提取能力；但这些类别与无人机任务无关，且模型无法输出 drone/bird 类别，因此需要在自己的数据集上微调。

4.3 无人机数据集与预处理

数据集来自题目推荐的 Roboflow **drone and bird**（版本 2，CC BY 4.0），共 2 类：bird（鸟）与 drone（无人机），图像已转为 YOLO 格式的 txt 标签。数据集划分与类别分布如下：

表 4-2 无人机数据集划分与类别分布

划分	图像数	bird 标注数	drone 标注数
train	3893	19306	1121
valid	649	4342	308
test	196	196	0

从分布可以看出两个重要问题：第一，**类别极不平衡**——训练集中 bird 标注 19306 个、drone 仅 1121 个，比例约 17:1；第二，**测试集只有 bird**（196 张图像，无 drone 标注），因此 test 指标只能反映鸟类的检测性能，不能代表无人机检测效果。后续分析以验证集（649 张，含 308 个 drone 标注）为主，并在报告中明确这一数据限制。

训练前对数据做了 YOLO 标准增强：HSV 颜色扰动（h=0.015, s=0.7, v=0.4）、随机平移 0.1、随机缩放 0.5、水平翻转 0.5、Mosaic 拼接 1.0，并在最后 10 个 epoch 关闭 Mosaic（close_mosaic=10），让模型在接近真实分布的数据上收尾。由于数据集中单图最多有 444 个目标（鸟群），Ultralytics 自动把 max_det 从 300 提升到 444，避免验证时截断导致召回率被低估。



图 4-2 训练集样例与 YOLO 标注框

4.4 模型训练

初始权重使用题目提供的 **drone.pt**（YOLO11n 结构，2 类 Bird/Drone，2.59 M 参数）。它在无人机数据上已有一定基础，因此采用较小学习率微调，而不是从头训练。训练配置如下：

表 4-3 YOLO11 无人机检测训练配置

配置项	取值
初始权重	data/task4/drone.pt（YOLO11n，2 类）
输入尺寸	640×640
Epochs	计划 50；实际训练 20 轮后提前停止
Batch size	16
优化器	AdamW，lr0=1e-3，余弦退火，weight_decay=5e-4
数据增强	HSV、平移、缩放、水平翻转、Mosaic（最后 10 轮关闭）
混合精度	AMP=True
设备	RTX 4060 Laptop 8 GB，workers=4，cache=disk
随机种子	42

训练在 8 GB 显存的 RTX 4060 上进行，batch=16、AMP 打开时显存占用约 2.5 GB，单轮训练约 30 – 35 秒。计划训练 50 轮，但实际观察到第 16 轮验证集 mAP@0.5 达到最高的 0.7396，此后在 0.70 – 0.74 之间波动、不再明显提升，因此在第 20 轮提前停止，取验证集最优的 best.pt 作为最终模型。训练过程中损失与 mAP 曲线由 Ultralytics 记录在 results.csv，本报告额外用 matplotlib 重新绘制了中文标注的损失/精度曲线。

4.5 训练结果与精度分析

表 4-4 YOLO11 无人机检测精度

指标	数值	备注
验证集 mAP@0.5	0.7307	—
验证集 mAP@0.5:0.95	0.3375	—
验证集 Precision	0.7614	—
验证集 Recall	0.7064	—

指标	数值	备注
验证集匹配框分类准确率	0.9989	—
测试集 mAP@0.5	0.6092	仅 bird 类别
测试集 mAP@0.5:0.95	0.3131	仅 bird 类别
测试集匹配框分类准确率	0.8583	仅 bird 类别

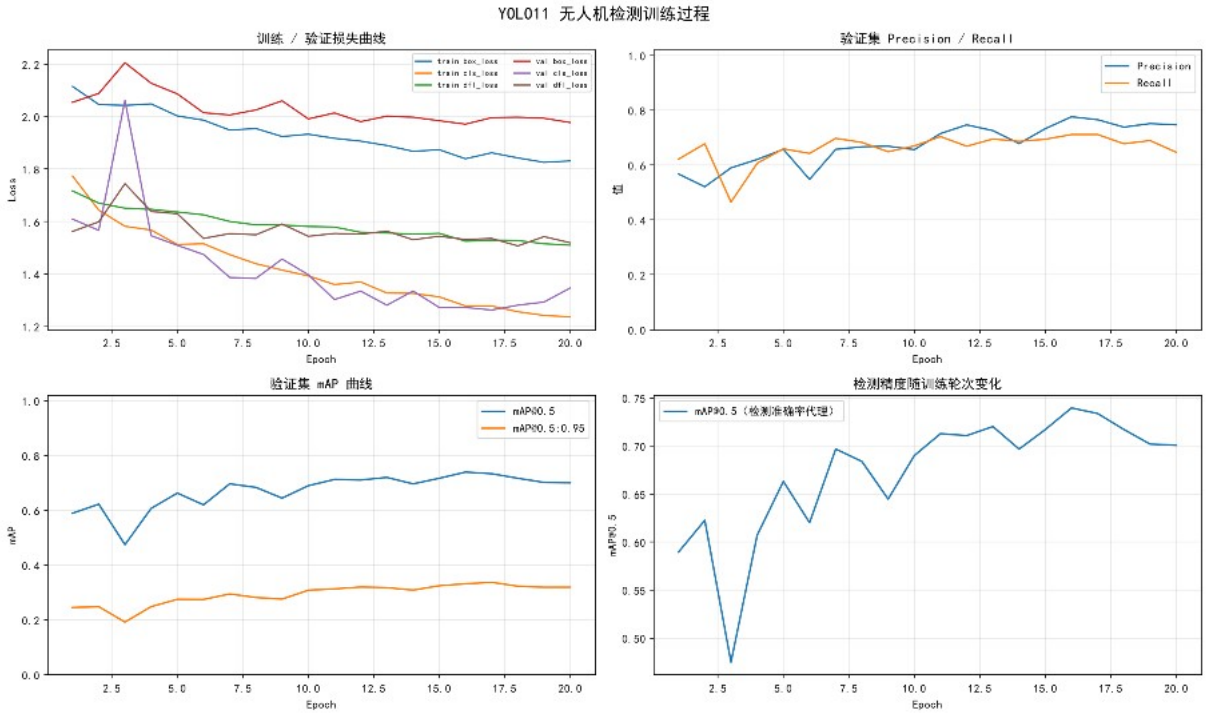


图 4-3 训练/验证损失、Precision/Recall 与 mAP 曲线

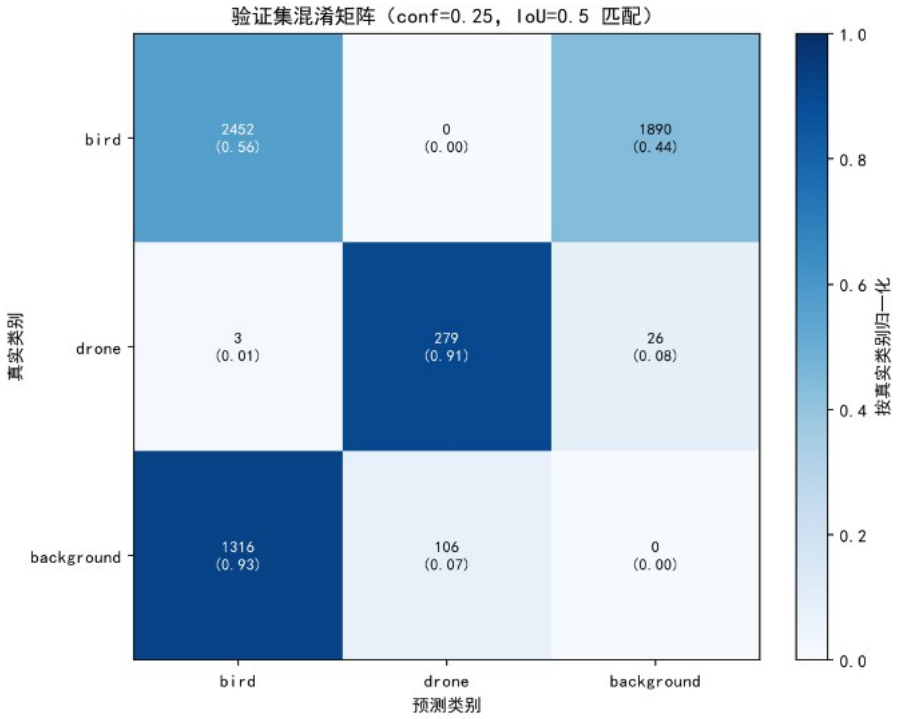


图 4-4 验证集混淆矩阵（conf=0.25，IoU=0.5 匹配；含漏检/误检）

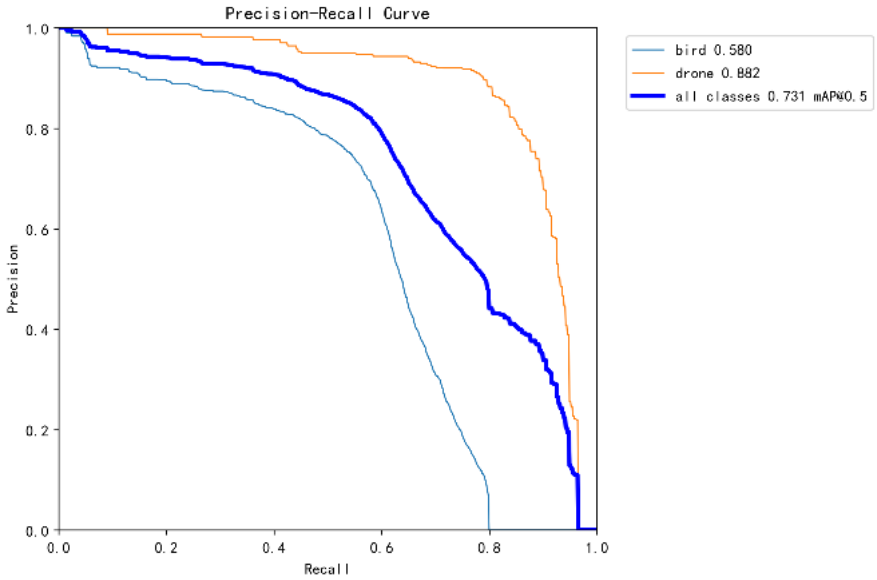


图 4-5 验证集 Precision－Recall 曲线

需要特别说明，检测任务不使用分类准确率（Accuracy）作为主要指标：一个检测框即使类别预测正确，只要位置偏移较大也不能算正确。因此本报告以 mAP 为核心指标，并额外计算“匹配框分类准确率”——先用 IoU≥0.5 把预测框与真值框贪心匹配，再统计匹配对中类别正确的比例。它回答的是“在已经定位到的目标里，类别判对了多少”，可作为题目要求的 Train Acc/Test Acc 的合理替代。

从混淆矩阵（conf=0.25、IoU=0.5 匹配）可以读出更具体的信息：bird 的召回率约 56.5%（2452/4342），drone 的召回率约 90.5%（279/308）；两者互相混淆很少（drone 被判成 bird 仅 3 个，bird 被判成 drone 为 0），说明类别判别本身并不困难，主要误差来自**漏检**——大量远处、密集的小鸟没有被检出；此外还有 1316 个 bird 误检和 106 个 drone 误检（背景列），多由建筑线条、天空纹理或相似轮廓造成。drone 召回率较高而 bird 召回率偏低，与 drone 样本少但目标更大、更显著有关；不能简单归因于类别不平衡。

4.6 视频推理与指标分析

使用微调后的 best.pt 对 example.mp4（1920×1080，24 fps，378 帧）推理，置信度阈值 0.25、NMS IoU 0.5、输入 640×640；同时用题目提供的 drone.pt 做对照。视频逐帧画框、置信度与目标中心轨迹，保存为 MP4。指标包括：检测覆盖率（有检测的帧占比）、平均置信度、目标框面积占比、相邻帧框 IoU、最长连续检测段、纯推理 FPS 与端到端 FPS。

表 4-5 example.mp4 视频推理指标对比

指标	drone.pt（初始）	best.pt（微调）
模型大小 / MB	5.23	25.20
检测覆盖率	100.00%	88.36%
检测类别（帧数）	Drone 99 / Bird 279	drone 311 / bird 23
平均置信度	0.6515	0.7480
平均目标框面积占比	48.477%	11.624%
相邻帧框平均 IoU	0.8738	0.9361
最长连续检测帧数	378	328
纯推理延迟 / ms	12.33	12.07
纯推理 FPS	81.1	82.9
端到端 FPS	27.5	35.9

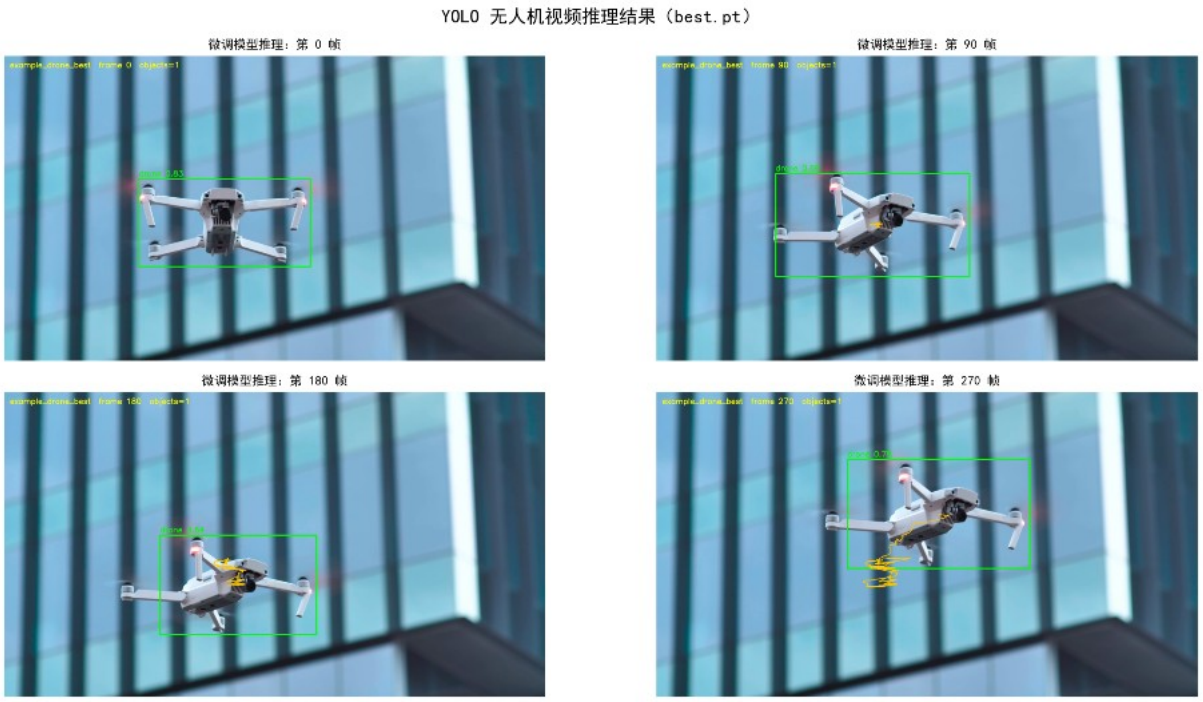


图 4-6 微调模型在视频关键帧上的检测结果

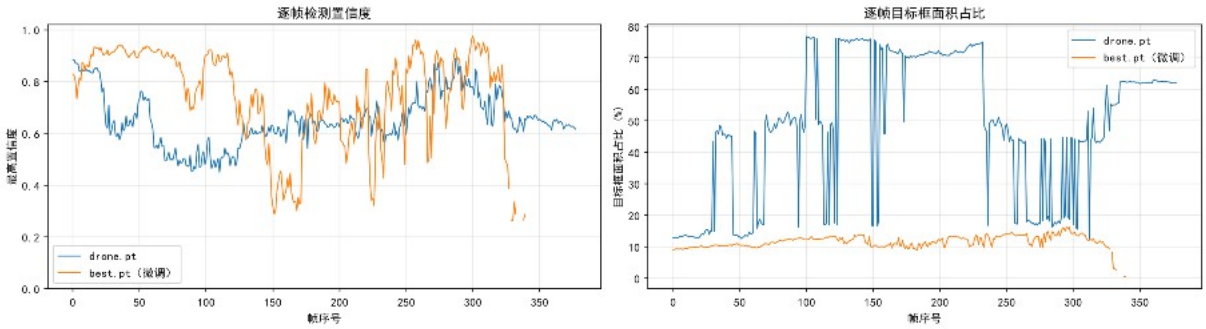


图 4-7 两个模型逐帧置信度与目标框面积占比

这组结果揭示了一个容易被“覆盖率”掩盖的问题：初始 drone.pt 的覆盖率虽然为 100%，但其检测类别是 **Drone 99 帧、Bird 279 帧**，大量帧把无人机判成鸟，并给出平均占画面 48% 的巨大框；微调后的 best.pt 则是 **drone 311 帧、bird 23 帧**，目标框面积占比降到 11.6%，相邻帧框 IoU 从 0.874 提升到 0.936。因此覆盖率从 100% 降到 88.4% 并不代表性能下降，而是初始模型用“鸟”这一高频类别和超大框“覆盖”了画面；微调模型在 44 帧（约占 11.6%）中没有给出置信度高于 0.25 的检测，主要出现在无人机快速运动/出画面的片段。

另外，best.pt 文件为 25.2 MB，这是因为 Ultralytics 检查点同时保存了模型、EMA 与优化器状态；实际部署时导出的 ONNX FP32 约 10 MB、FP16 约 5 MB、INT8 约 3.0 MB（见 4.7 节）。初始 drone.pt 的 5.23 MB 是纯推理权重，两者不能直接比较文件大小。

题目的指标中还包括 EAO（Expected Average Overlap，期望平均重叠率）。EAO 是 VOT 跟踪数据集的标准指标，需要每一帧的真值框，按跟踪序列长度求平均 IoU。本实验的视频没有逐

帧标注，因此无法严格计算 EAO；作为替代，报告给出相邻帧检测框的平均 IoU 与最长连续检测段，用来衡量“跟踪的连续性/稳定性”。若后续补充真值，可直接用同样的检测框序列计算 EAO。

端到端 FPS 包含视频解码、预处理、推理、后处理与编码写盘，因此明显低于纯推理 FPS；实际部署时应把解码/编码放到独立线程，或使用 TensorRT、GPU 端到端流水线提升吞吐量。微调模型与初始模型的推理速度接近（结构相同、参数量相同），差异主要来自检测框数量与后处理耗时。

4.7 FP16 / INT8 量化与 ONNX 导出

为满足“在保证精度的前提下提高推理速度”的提高部分要求，实验将 best.pt 导出为 ONNX，并进一步做 FP16 与 INT8 量化：

- **FP32 ONNX**：`model.export(format='onnx', imgsz=640, half=False, simplify=True, opset=13)`，作为量化基线。
- **FP16 ONNX**：导出时设置 `half=True`，权重与计算用 FP16 存储，模型体积约减半，适合 GPU / TensorRT 部署。
- **INT8 ONNX**：用验证集 120 张图片做 MinMax 校准，采用 QDQ 格式、per-channel 权重量化、对称/非对称激活量化，通过 ONNX Runtime `quantize_static` 生成静态量化模型。

量化后在同一验证集上评估 mAP@0.5，并在 640×640、batch=1 下做延迟基准（warmup 15 次，取平均）。结果如下：

表 4-6 ONNX 导出与 FP16/INT8 量化对比（PyTorch 为 GPU，ONNX 为 CPU 8 线程）

模型	大小 / MB	mAP@0.5	延迟 / ms	FPS
PyTorch best.pt（检查点）	25.20	0.7311	14.73	67.9
ONNX FP32	10.11	0.7201	19.96	50.1
ONNX FP16	5.11	0.7205	21.31	46.9
ONNX INT8	3.05	0.6576	17.84	56.1

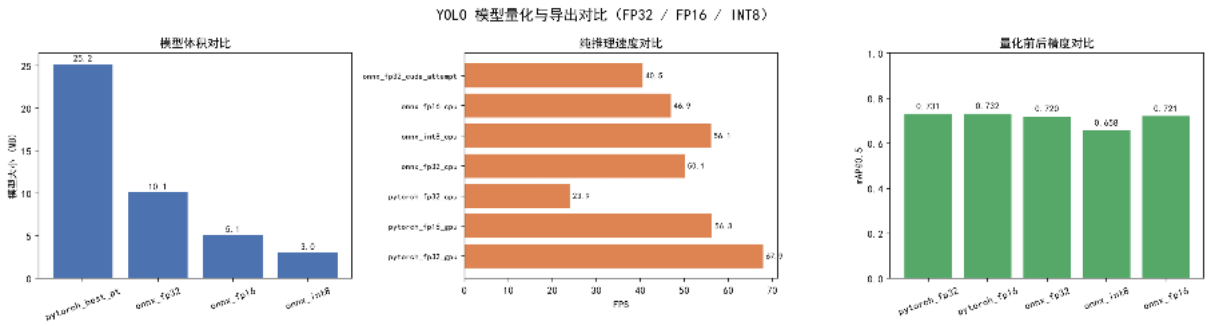


图 4-8 模型体积、推理速度与量化前后精度对比

量化带来的收益与代价在本实验中体现得很典型。**FP16** 把权重从 32 bit 降到 16 bit，模型体积从 10.11 MB 降到 5.11 MB（约 50.5%），mAP@0.5 为 0.7205，与 FP32 的 0.7201 基本一致，说明 FP16 在几乎不损失精度的情况下减半了体积。但本机 ONNX Runtime 的 CUDA Provider 需要 CUDA 13.x，而 PyTorch 使用的是 CUDA 12.8，加载时因缺少 cublasLt64_13.dll 回退到 CPU，因此 FP16 ONNX 的GPU 速度无法在本环境测出；CPU 上 FP16 反而比 FP32 慢（21.3 ms vs 20.0 ms），因为 CPU 缺少高效 FP16 卷积内核。FP16 的加速需要 TensorRT 或匹配版本的 ORT-GPU，这是部署环境依赖的典型例子。

INT8 采用 QDQ 静态量化、per-channel 权重、MinMax 校准（120 张验证图），模型体积 3.05 MB，仅为 FP32 的 30.2%；mAP@0.5 从 0.7201 降到 0.6576（相对下降 8.7%），CPU 延迟从 19.96 ms 降到 17.84 ms（约 1.12 倍，50.1→56.1 FPS）。加速幅度不大，原因是 AMD Ryzen 9 7945HX 上 ORT 的 INT8 卷积内核利用率有限，且 QDQ 的 Quantize/Dequantize 节点带来额外开销；在支持 VNNI/TensorRT INT8 的平台上通常能获得更大的加速。

量化过程中还遇到一个关键工程问题：YOLO 的输出是一个 Concat，把范围 0–640 的框坐标与范围 0–1 的类别分数拼在一起；若直接量化该 Concat，二者共享激活尺度，类别分数会被量化成 0，mAP 直接归零。本实验通过 `nodes_to_exclude` 排除输出 Concat 节点，保留其浮点输出，才得到可用的 INT8 模型。这说明量化不能只看“模型能不能跑”，必须逐层检查输出分布与敏感层。

若精度下降不可接受，可采用混合量化：只量化 Backbone/Neck，保留 Head 为 FP16；或增加校准图片数量、采用 percentile 校准、对类别分支使用更高位宽。若追求部署速度，应优先使用 TensorRT FP16/INT8 或厂商 NPU 工具链，而不是在通用 CPU 上依赖 ONNX Runtime 的 INT8 内核。

4.8 误差分析与改进方向

结合训练曲线、混淆矩阵与视频推理结果，模型的主要误差来源包括：

- **类别不平衡与目标尺度**：bird 标注约是 drone 的 17 倍，但 bird 目标通常更小、更密集，导致 bird 漏检多（召回率约 56.5%）；drone 虽然样本少，但目标更大、更显著，召回率反而达到约 90.5%。改进重点应放在小目标与密集场景（提高输入分辨率、SAHI 切片、增加小目标样本），而不是简单重采样。
- **小目标与运动模糊**：视频中的无人机只占画面很小区域，640 输入下下采样后细节更少。可提高输入分辨率（如 960/1280）、使用 SAHI 切片推理、或增加多尺度训练。
- **背景干扰**：玻璃幕墙、建筑线条与天空的高频纹理容易产生误检；可增加负样本（无目标的背景图）、使用更强的颜色/几何增强。
- **测试集偏差**：test 只有 bird，无法评估 drone；应在验证集上按类别分别报告 mAP，并补充 drone 的独立测试集。
- **视频后处理**：逐帧检测没有利用时间信息，可通过 ByteTrack/BoT-SORT 做多目标跟踪，用轨迹平滑、置信度投票与丢失补偿提升视频稳定性，这也是计算 EAO 的前提。

在部署层面，可进一步尝试 TensorRT 导出（FP16/INT8）、ONNX Runtime GPU provider、批处理与 CUDA 预处理，把端到端 FPS 从当前的几十帧提升到上百帧；同时用 INT8 校准集覆盖更多场景（不同光照、背景、目标尺度），减小量化误差。

5 总结

本项目按题目要求完成了 task-2、task-3 与 task-4 题目一。task-2 从最基础的读图、通道分离、几何变换出发，逐步实现颜色先验轮廓检测、MOG2/帧差运动检测与三种相关滤波跟踪器，并对 example4 的失败案例做了定量分析与改进尝试；task-3 把深度学习概念落到两个可运行的实验上：用 autograd + 梯度下降拟合含指数项的二元函数（R^2=0.9516），以及用 MLP 在 MNIST 上达到 98.62% 的测试准确率；task-4 完成了从官方预训练推理、数据集分析、drone.pt 微调、视频指标分析到 ONNX/FP16/INT8 量化的完整链路。

过程中体会最深的三点：第一，**数据质量决定上限**。轮廓检测的难点在分割，检测任务的瓶颈在类别不平衡与测试集偏差，模型结构反而不是第一位；第二，**指标要与任务匹配**。检测不能只看 accuracy，跟踪不能只看“框有没有画出来”，没有真值时要明确替代指标的局限性；第三，**工程细节影响可复现性**。中文路径、Windows DataLoader、ONNX 导出时的 half/opset、量化校准集选择，任何一个小问题都可能让实验失败，版本控制与日志记录非常重要。

后续可以从三个方向继续深入：一是用更强的检测/跟踪模型（YOLO11s/m、ByteTrack、BoT-SORT）提升无人机视频的召回与稳定性；二是补齐逐帧标注，严格计算 EAO、成功率、Precision 等跟踪指标；三是把量化模型部署到 Jetson/边缘设备，测试真实功耗与端到端延迟。

附录 A 项目结构与运行方法

```
cv-recruitment/
├─ data/                          # 题目素材与数据集（不纳入 git）
│  ├─ task2/  task3/  task4/
│  └─ drone_dataset/              # Roboflow YOLO 数据集
├─ task2_opencv_pillow/scripts/   # task2_image_ops / contours / motion / tracking
├─ task3_mlp_mnist/src/           # least_squares_fit / mnist_mlp
├─ task4_yolo_drone/src/          # 01_pretrained / 02_train / 03_video / 04_quantize
├─ report/                        # 本报告与图表
├─ requirements.txt
└─ README.md
```

所有脚本均可从项目根目录直接运行：

```
python task2_opencv_pillow/scripts/task2_image_ops.py
python task2_opencv_pillow/scripts/task2_contours.py
python task2_opencv_pillow/scripts/task2_motion.py
python task2_opencv_pillow/scripts/task2_tracking.py
python task3_mlp_mnist/src/least_squares_fit.py
python task3_mlp_mnist/src/mnist_mlp.py --epochs 20
python task4_yolo_drone/src/01_pretrained_inference.py
python task4_yolo_drone/src/02_train.py --epochs 50 --batch 16
python task4_yolo_drone/src/03_video_inference.py
python task4_yolo_drone/src/04_export_quantize.py
```

附录 B 提交材料清单

- task-2：源码；example2/3/4 的检测与跟踪视频；通道分离、几何变换、轮廓标注、MOG2/帧差对比、跟踪轨迹等图片；本报告第 2 章。
- task-3：源码；最小二乘拟合图与参数/R^2；MLP 的 Loss、Train Acc、Test Acc 曲线、混淆矩阵与预测可视化；本报告第 3 章。
- task-4：源码；COCO 预训练推理图片；drone.pt 微调的 best.pt 与训练曲线/混淆矩阵；example.mp4 推理视频与指标；ONNX FP32/FP16/INT8 模型与量化对比；本报告第 4 章。

说明：数据集、模型权重与视频体积较大，未纳入 git；报告中的图表均由脚本自动生成，可用 `python report/build_report.py` 复现本文档。